

# CalBot

## Inf2007 Mobile Application Development Spring 2026

If you copy from your peers, you aren't learning, and you violate the principles of academic integrity.

For this quiz, you should avoid referencing/copying their code. However, consulting online resources is OK.

Fork the repo **inf2007-quiz01-2026**. The Function .kt file: **FuntionMap.kt** and **MainActivity.kt** are in the Xsite.

A CalBot is software that simulates calculator-like interaction with users. In this quiz, you are going to a CalBot app. In this app, the user first selects a CalBot from a list (hard-coded) to navigate to the calculator screen to calculate with the selected CalBot. All CalBots act the same, they use Room Database/Preference DataScore to store all the history and preference and provides responses for computing a simple function.

In the calculator screen, the user hits the number and operator buttons ("+", "-", "\*", "/") to input an arithmetic expression and hit "=" to compute and display the result. The user can choose to toggle the URL option, then the computation would be performed by math.js API (<https://api.mathjs.org/>) through **Retrofit**. This toggle setting and the expression history (a maximum of 20 expressions each calculator) will be stored persistently, using the **Preferences DataStore** and **Room Database**.

## Refine the UI

This app has one activity: **MainActivity** and two screens. Note that the screens should maintain similarity in layout, relative positions, and sizes, but an exact 1:1 duplication is **NOT** required. The views need to be **visible after rotation**, e.g. using weight.

### In the Contact Screen

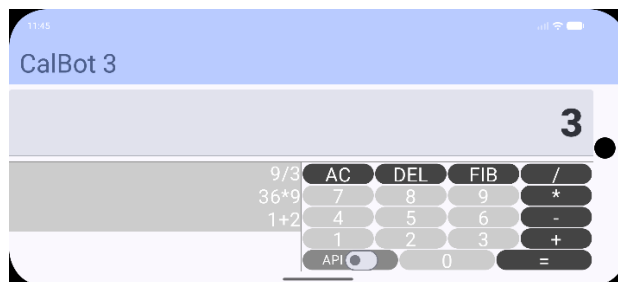
- A **LazyColumn** (testTag: **calBotList**) has to be used to display the list of 30 **CalBots**. This list is **hardcoded**, their names are **CalBot 1** to **CalBot 30**.
- Each item in the **LazyColumn** has to be a **Card** (testTag: **calBotItem**) which is clickable (hit to navigate to the calculator screen to compute with the selected CalBot)

- This **Card** contains a **Row** of **Image** (testTag: **calBotIcon**) and **Text** (testTag: **calBotName**). The **Image** uses the *R.drawable.ic\_launcher\_foreground* as the *painterResource* and the **Text** displays the name of the **CalBot**.



### In the Calculator Screen

The basic UI of this screen is given in **Xsite, MainActivity.kt**, it has three parts: **display** (displaying the input expression and result), **history** (display the expression history) and **calculator pad** (input buttons). Examine the given code and update the UI accordingly. Currently, the UI works in Portrait mode, fix the UI so all the widgets are visible after rotation to Landscape mode. Note that, the history (**testTag**: history) would be arranged to the left of the calculator pad. The screen to be similar to the following.



**IMPORTANT: The texts and testTags are given in the code, don't change them.**

## Implement the functionality

### 1. Implementing the navigation

When a CalBot **Card** is selected, its ID or name can be passed to the Calculator Screen through navigation. Additionally, when navigating back from the Calculator Screen, the displayed CalBot list may be updated (as mentioned in the last section). This can be implemented using either one shared ViewModel that contains all CalBot histories, with filtering applied based on the CalBot selected, or per-screen ViewModels.

- Using a Shared ViewModel: One approach can be holding all history, and apply mapping/filtering to retrieve the messages of the selected CalBot.
- Using per-screen ViewModels: You may need to have ViewModelFactory to handle to parameter that get from the navigation like the CalBot ID.
- Using Hilt with per-screen ViewModels: An easier approach is to use a method-based factory, set the calBot ID and refetch the data after the hiltViewModel is created.

### 2. Implement the basic arithmetic computation

- The user hits the number ("0-9") and operator buttons ("+", "-", "\*", "/") to input an arithmetic expression that is displayed in the **TextField** (**testTag**: display).
- The user hits "=", to compute the result, and the result will replace the expression and is displayed in the **TextField** (**testTag**: display), the result can be reused as the next expression.
  - Note that Kotlin cannot directly evaluate an expression, you need to handle the 4 operator cases respectively.
  - You can assume the inputs are simple Integers and the result is within the Integer range.
- Implement **AC** logic to clear the display.
- Implement **DEL** logic to backspace the one digit or operator.
- Note that you only need to handle two numbers and one operator, that is number1 operator number2, e.g., 1+1, and assume all the expressions are valid with no overflow and are of simple **Integers**.

### 2. Implementing the arithmetic computation with math.js API

- If the API is toggled, the app queries the **math.js API** (<https://api.mathjs.org/>) using Retrofit to evaluate the input expression and display the response result as the same as the basic arithmetic computation.
- This REST interface returns only one number, thus you do not need to parse it from JSON. You can use the following interface:

```
@GET("?")
suspend fun calculate(@Query("expr") expr: String): Response<Number>
```

and use a similar builder as the example given in class:

```
var retrofit = Retrofit.Builder()
    .baseUrl("https://api.mathjs.org/v4/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
```

and get the response using:

```
calculate(expr).body()
```

- Usually, a coroutine with an appropriate Dispatcher is needed to perform the query.

### 3. Implementing the persistent history and preference setting.

- The expression history (excluding the results) **of the selected CalBot** will be maintained and displayed in **LazyColumn** (**testTag**: history) with a maximum of 20 expression.
- It places and shows the most recent input expression at the top of the **LazyColumn**, discarding the oldest if it surpasses the maximum limit (20).
- The API toggle status (**testTag**: toggleAPI) and the history will remain persistent after app relaunch, you can use Preference Datastore to implement this.

### 4. Implementing calling the Fibonacci number computation

- If the user hit **FIB**, then the computation of Fibonacci number is required. This computation returns an Int and takes the input number, use the **FunctionMap.kt** in **Xsite**.
- Note that: once **FIB** (not "=") is hit, the computation starts
- E.g., hit 6 and **FIB** will call Fibonacci function fib(6) and return 8:
  - If hitting hit 6 and FIB by the user, the history "fib(6)" will be **immediately added** to history avoiding ANR or blocking the main UI, e.g., the user can continue toggle API.
  - This computation will be dispatched to another thread (you can choose to use Coroutine or Thread or WorkManager to implement this), after the result is computed, it is displayed as normal.
- A map of high-order functions is provided in the FunctionMap.kt (the slow function self or fib(44) can be used for testing), the function fib can be referenced by using FunctionMap. functionMap[key] with key="fib".

## 5. Implement back from compute

- After pressing the system back button in the Calculator screen:
  - If you did not compute with the selected CalBot, nothing needs to be handled.
  - However, if you have computed with the CalBot, a **"=" has been pressed**, then this CalBot (most recent one) will be moved to the top of the CalBot list after pressing the system back button.
  - This does **NOT** need to be persistent after app relaunch, however, it needs to survive from app rotation.
- After Back and Reselect: if the new CalBot is different, the **display** is cleared.

## Programming Quiz

**Submission deadline: 19 Mar 2026, 9:00 am**

(Total 14 Marks)

Fork the repo **inf2007-quiz01-2026**.

1. Refine the UI similar to the given screenshots. The widgets need to be visible after rotation. (1 Mark)
2. Implement the basic arithmetic computation. (1 Mark)
3. Implement DEL and AC functions. (1 Mark)
4. Implement the display of the expression history. (1 Mark)
5. Implement toggle API and computation using mathjs with Retrofit. (2 Mark)
6. Implement the computation of Fibonacci number without blocking the main UI. (2 Marks)
7. Implement the persistent history and toggle API. (2 Marks)
8. Implement the navigation, only the selected CalBot's history is shown. (1 Mark)
9. Implement back from calculator, the most recent computed CalBot will be moved to the top of the CalBot list, the **display** will be cleared when select a new CalBot. (1 Mark)
10. Remember to comment and indent the code and implement it in a modular fashion, using different methods or classes if appropriate. **Update the README.md about what you have done or partially done**, i.e., those 1-11 features.: 1. Done, 2. Done, 3. Partly Done ... (2 Marks)
11. Commit and push all changes to your forked repository **inf2007-quiz01-2026**.  
**The Function .kt file: FunctionMap.kt** needs to be committed. Ensure the code is compilable; failure to compile or commit to wrong repository could result in 0 marks.
12. You may also use the **test folder** (which includes the test APK) from XSite to validate your solution. Place the test folder at the project root (the same level as README.md). Note that it is optional to commit these test files.

**IMPORTANT: Do not change the activity name or package name. Ensure there are no spelling errors on all the text and testTags. Remember to commit and push all files to the right repo!**

**END OF DOCUMENT**